

Reviewer Pack — Minimal Rank of Tropicalized Algebraic K-theory over AC^0 -k-D...

Entry 0d591e12592c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 09:20:01 UTC

Minimal Rank of Tropicalized Algebraic K-theory over AC^0 -k-Distance Circuit Size

Entry ID: 0d591e12592c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 09:20:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory (Tropicalization) **Field B** (complexity object): Complexity Theory: AC^0 -k-Distance Circuit Complexity

Statement:

[‘For every AC^0 -k-distance circuit C with $k < \log_2 n$, the minimal rank of its tropicalized algebraic K-theory group is at least $\Omega(n^{\{1/3\}})$ and at most $O(2^n)$.’, ‘Equivalently, for all AC^0 -k-distance circuits C computing a function f on n inputs, the rank of the induced tropicalized algebraic K-theory group $T(K(C))$ satisfies $\text{rank}(T(K(C))) \geq \Omega(n^{\{1/3\}})$ and $\text{rank}(T(K(C))) \leq O(2^n)$.’, ‘This is true for all AC^0 -k-distance circuits C regardless of whether they compute f correctly or not.’]

Rationale (proposer’s reasoning):

[‘Algebraic K-theory provides a rich algebraic structure that can capture the complexity of computational problems, while tropicalization allows us to study these structures in a more geometric and combinatorial way.’, ‘Previous work has shown that tropicalized algebraic objects can reveal insights into the complexity of computation. This conjecture explores the potential of using tropicalized algebraic K-theory to understand AC^0 -k-distance circuit complexity.’, ‘If this conjecture is true, it would provide a new tool for studying the complexity of circuits and could potentially lead to improved lower bounds.’]

Taxonomy category: AC^0 _PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dda5d46758362cb0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The rank of tropicalized algebraic K-theory group $T(K(C))$ from a random AC^0 -k-distance circuit C with $k < \log_2 n$ and $n \leq 40$, computed using 30 seeds, must be within bounds: $\geq \Omega(n^{\{1/3\}})$ AND $\leq O(2^n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropicalization Algebraic K-theory' AND 'AC⁰-k-distance circuit complexity' - 'minimal rank tropicalized algebraic K-theory' AND 'AC⁰ circuit size' - rank T(K(C)) ' $\Omega(n^{\{1/3\}})$ ' ' $O(2^n)$ ' AC⁰-k-distance

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        # Simplified AC0-k-distance circuit generation for demonstration
        return [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]

    def algebraic_K_theory(C):
        n = len(C)
        K_C = []
        for i in range(n):
            row_sum = sum(C[i][j] * C[j][k] for j in range(n) for k in range(n))
            K_C.append(row_sum)
        return K_C

    def tropicalize(A):
        m, n = len(A), len(A[0])
        T = [[max(row[j] for row in A) if col == j else float('-inf') for j in range(n)] for i in range(m)]
        return T

    def rank(matrix):
        m, n = len(matrix), len(matrix[0])
        matrix_copy = [row[:] for row in matrix]
        rank = 0
        for i in range(min(m, n)):
            if all(x == float('-inf') for x in matrix_copy[i]):
```

```

        continue
    pivot_row = max(range(i, m), key=lambda r: matrix_copy[r][i])
    if matrix_copy[pivot_row][i] == float('-inf'):
        continue
    rank += 1
    for j in range(n):
        matrix_copy[i][j], matrix_copy[pivot_row][j] = matrix_copy[pivot_row][j], matrix_copy[i][j]
    for r in range(m):
        if r != i:
            factor = matrix_copy[r][i] / matrix_copy[i][i]
            for j in range(n):
                matrix_copy[r][j] -= factor * matrix_copy[i][j]

    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    K_C = algebraic_K_theory(generate_circuit(n))
    T_K_C = tropicalize(K_C)
    rank_T_K_C = rank(T_K_C)
    results.append({
        "n": n,
        "rank": rank_T_K_C
    })

metric_value = sum(result["rank"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(2**(1/3) * n <= result["rank"] <= 2**n for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Rank of Tropicalized Algebraic K-theory",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(seed) for seed in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 10)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bcdb8697.py", line 90, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bcdb8697.py", line 63, in run_trial
    T_K_C = tropicalize(K_C)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bcdb8697.py", line 34, in tropicalize
    m, n = len(A), len(A[0])
             ^^^^^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's validity. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10396
2	propose	ollama_remote	glm4:latest	0	0	12200
3	preregistration	ollama_remote	glm4:latest	0	0	6086
4	novelty	ollama_remote	glm4:latest	0	0	4888
5	novelty	ollama_remote	glm4:latest	0	0	5483
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14724
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8142
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12633

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11132
10	judge	ollama_remote	glm4:latest	0	0	8384

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94068 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/0d591e12592c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0d591e12592c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0d591e12592c.tar.gz (if generated)